> **Fuente original:** https://algoritmos-rw.github.io/tda_bg/material/guias/dyc/

# División y Conquista

## Contenidos


1. Ejercicio resuelto
2. Ejercicios propuestos


# Ejercicio resuelto

## Enunciado

Dado un arreglo de n enteros (no olvidar que pueden haber números negativos), encontrar el subarreglo contiguo de máxima suma, utilizando División y Conquista. Indicar y justificar la complejidad del algoritmo. Ejemplos:

```
[5, 3, 2, 4, -1] →  [5, 3, 2, 4]
[5, 3, -5, 4, -1] →  [5, 3]
[5, -4, 2, 4, -1] → [5, -4, 2, 4]
[5, -4, 2, 4] → [5, -4, 2, 4]
```

## Consideraciones Iniciales

En primer lugar, conviene analizar cuál es la forma que tienen las posibles respuestas al enunciado. Se tratan en este caso de partes del arreglo completo de mayor o menor longitud. Solo se pueden seleccionar elementos contiguos, por lo que los subarreglos posibles serían:

- El primer elemento solo
- El primero y el segundo
  - Y así hasta el último siendo $n$ subarreglos que empiezan con el primero
- El segundo solo
- El segundo y el tercero
  - Y así hasta el último siendo $n - 1$ subarreglos que empiezan con el segundo
- Y así siguiendo con los n elementos

La cantidad total de soluciones posibles sería entonces $n + (n - 1) + \cdots + 1$ , lo cual se puede escribir como:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

La demostración formal no va al caso, pero se puede recordar aquí. Si quisiéramos obtener la mejor solución simplemente comparando cada solución posible, la complejidad temporal del algoritmo sería $\mathcal{O}(n^2)$.

Cabe destacar también que el algoritmo debe permitir el uso de números negativos. De lo contrario, el subarreglo de máxima suma siempre sería el arreglo completo. Si todos son positivos, nada va a sumar más que todos los elementos del arreglo.

## Caso Base

Ahora bien, si el arreglo tiene un solo elemento, la solución es simplemente usar ese elemento sea cual sea su valor. Cuando son dos, puede convenir más usar solo el primero (`[1,-1]`), solo el segundo (`[-1,1]`) o ambos (`[1,1]`) dependiendo del caso. Si son más de dos elementos, se va complicando todo.

## División y Conquista

Utilizando el caso base de un elemento, podemos dividir el arreglo por la mitad hasta obtener el caso en que tenemos que comparar dos subarreglos de largo 1.

Pero además de eso, debemos revisar cómo se comparan los arreglos generados por la recursión en cada llamada. Tenemos los elementos del lado izquierdo y del lado derecho de la llamada. Podríamos comparar el subarreglo candidato de cada lado y devolver el que sume más.

Sin embargo esto no es suficiente. En cada llamada, existen subarreglos candidatos que se pueden formar con los elementos del lado izquierdo y derecho, siempre y cuando sean contiguos.

Para aclarar esto, supongamos que en una llamada recursiva tenemos `[1,2]` en el arreglo derecho y `[3,-5]` en el lado izquierdo. El mejor subarreglo del lado izquierdo es `[1,2]` y el mejor del lado derecho es `[3]`. Pero para el arreglo combinado (`[1,2,3,-5]`) el mejor subarreglo es `[1,2,3]`.

Eso significa que en cada caso recursivo la mejor solución puede usar solo elementos del lado izquierdo, solo elementos del lado derecho o una combinación de elementos de ambos.

Es una situación similar al ejercicio de puntos más cercanos donde tenemos dos alternativas por la recursión pero hay que tener en cuenta que pasa al medio para asegurar la mejor decisión

## Máximo Subarreglo Cruzado

Entonces, dada una llamada recursiva con el mejor candidato del lado izquierdo y derecho, necesitamos obtener el mejor candidato que use elementos de los arreglos a ambos lados de la recursión y que además sea contiguo. A este subarreglo candidato se lo suele llamar subarreglo cruzado. Solo basta con obtener el subarreglo cruzado que sume más.

Para obtener el máximo subarreglo cruzado, podemos aprovecharnos de que tiene que ser contiguo y hacer lo siguiente.

Primero iteraremos el arreglo del lado izquierdo desde su índice mayor hasta el menor. La idea es ir sumando los elementos en una variable e ir guardando el índice que hace que la suma sea máxima. Es necesario que se iteren todos los elementos del arreglo izquierdo pues no se puede asegurar cuál índice será el que maximice la suma.

Se hace lo mismo, pero con el arreglo derecho desde el índice menor hasta el mayor.

Luego, el arreglo cruzado de suma máxima será el arreglo izquierdo y derecho combinados desde el índice que maximiza la suma a izquierda hasta el índice que maximiza la suma a derecha. El subarreglo candidato es contiguo y está formado por las partes de cada lado que suman más.

Teniendo el candidato a izquierda, a derecha y el máximo subarreglo cruzado, se debe comparar cual suma más y devolver esa solución para esa llamada recursiva.

## Implementación

```python
def subarreglo_cruzado(arr):
    med = len(arr) // 2

    suma_parcial = 0
    cruzado_izq = med
    suma_izq = 0
    for i in range(med - 1, -1, -1):
        suma_parcial += arr[i]
        if suma_parcial > suma_izq:
            suma_izq = suma_parcial
            cruzado_izq = i

    suma_parcial = 0
    cruzado_der = med
    suma_der = 0
    for j in range(med, len(arr)):
        suma_parcial += arr[j]
        if suma_parcial > suma_der:
            suma_der = suma_parcial
            cruzado_der = j

    return arr[cruzado_izq : cruzado_der + 1]

def subarreglo_maxima_suma(arr):
    if len(arr) == 1:
        return arr

    med = len(arr) // 2

    izq = subarreglo_maxima_suma(arr[:med])
    der = subarreglo_maxima_suma(arr[med:])
    cruzado = subarreglo_cruzado(arr)

    sumas = (sum(izq), sum(der), sum(cruzado))
    suma_maxima = max(sumas)

    if suma_maxima == sumas[0]:
        return izq
    elif suma_maxima == sumas[1]:
        return der
    else:
        return cruzado
```

## Complejidad

Para justificar la complejidad, utilizamos el teorema maestro. Se realizan dos llamadas recursivas para obtener el candidato a izquierda y derecha por lo que $A = 2$. En cada llamada recursiva se divide el arreglo por la mitad, por lo tanto $B = 2$. Para calcular el arreglo cruzado se deben recorrer ambas mitades hasta el final del ciclo haciendo que sea lineal en función de la cantidad de elementos. Calcular la suma de los subarreglos para comparar también es lineal, siendo el peor caso la suma de un subarreglo que contenga a todos los elementos. Eso significa que los cálculos auxiliares tienen complejidad lineal, resultando en que $C = 1$.

La ecuación de recurrencia queda entonces como:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Haciendo los cálculos correspondientes llegamos a que

$$\log_2(2) = 1 = C \implies \mathcal{O}(n \cdot \log(n))$$

Es interesante notar que con el enfoque de división y conquista, se aprovechan los cálculos intermedios para poder obtener la solución sin necesidad de probar todas las combinaciones.

# Ejercicios propuestos

1. (★) Se cuenta con un arreglo de enteros ordenado de manera ascendente que contiene exactamente un número duplicado (es decir, todos los demás elementos son distintos, sin duplicados). Implementar una función que encuentre dicho número utilizando división y conquista. Indicar y justificar la complejidad del algoritmo, utilizando el Teorema Maestro.

2. (★) Se tiene un arreglo en el que se registran los resultados de tests automáticos de una porción de código. Este código se encontraba funcionando pero, debido a unos cambios que se están realizando, en algún momento dejó de funcionar. Se registra un 1 si pasa los tests, 0 en caso contrario. De esta manera, el arreglo tendrá la forma `[1, 1, 1, ..., 0, 0, ...]` (es decir, *unos seguidos de ceros*). Se pide: a. una función de orden $\mathcal{O}(\log n)$ que, por división y conquista, encuentre el índice del primer 0, de forma que se pueda reconocer rápidamente en qué modificación del código se dejó de pasar los tests. Si no hay ningún 0 (solo hay unos), debe devolver -1. b. demostrar con el Teorema Maestro que la función es, en efecto, $\mathcal{O}(\log n)$.

   Ejemplos:

   ```
   [1, 1, 0, 0, 0] →  2
   [0, 0, 0, 0, 0] →  0
   [1, 1, 1, 1, 1] → -1
   ```

3. (★) Implementar un algoritmo que, por división y conquista, permita obtener la parte entera de la raíz cuadrada de un número $n$, en tiempo $\mathcal{O}(\log n)$. Por ejemplo, para $n = 10$ debe devolver 3, y para $n = 25$ debe devolver 5. Justificar la complejidad del algoritmo.

4. (★) Se tiene un arreglo de $N >= 3$ elementos en forma de pico, esto es: estrictamente creciente hasta una determinada posición $p$, y estrictamente decreciente a partir de ella (con $0 < p < N - 1$). Por ejemplo, en el arreglo `[1, 2, 3, 1, 0, -2]` la posición del pico es $p = 2$. Se pide: a. Implementar un algoritmo de división y conquista de complejidad $\mathcal{O}(\log n)$ que encuentre la posición $p$ del pico. b. Justificar la complejidad del algoritmo mediante el teorema maestro.

5. (★) Implementar Merge Sort. Justificar la complejidad del algoritmo mediante el teorema maestro.

6. (★) Implementar un algoritmo de multiplicación de dos números grandes de longitud n, por división y conquista, con un orden de complejidad mejor que $\mathcal{O}(n^2)$. Justificar la complejidad del algoritmo mediante el teorema maestro.

7. (★★) Implementar un algoritmo que dados n puntos en un plano, busque la pareja que se encuentre más cercana, por división y conquista, con un orden de complejidad mejor que $\mathcal{O}(n^2)$. Justificar la complejidad del algoritmo mediante el teorema maestro. Se puede asumir que ningún par de puntos tienen la misma coordenada x o y.

8. (★) Dados un conjunto de n elementos, y 2 arreglos de longitud n, con dichos elementos. El arreglo A está completamente ordenado de menor a mayor. El arreglo B se encuentra desordenado. Indicar, por división y conquista, la cantidad de inversiones necesarias al arreglo B para que quede ordenado de menor a mayor, con un orden de complejidad mejor que $\mathcal{O}(n^2)$. Justificar la complejidad del algoritmo mediante el teorema maestro.

9. (★★★) Implementar una función, que utilice división y conquista, de complejidad $\mathcal{O}(n \log n)$ que dado un arreglo de $n$ números enteros devuelva `true` o `false` según si existe algún elemento que aparezca más de la mitad de las veces. Justificar la complejidad de la solución. Ejemplos:

   ```
   [1, 2, 1, 2, 3] -> false
   [1, 1, 2, 3] -> false
   [1, 2, 3, 1, 1, 1] -> true
   [1] -> true
   ```

   *Aclaración*: Este ejercicio puede resolverse, casi trivialmente, ordenando el arreglo con un algoritmo eficiente, o incluso se puede realizar más rápido utilizando una tabla de hash. Para cumplir con la consigna, resolver **sin ordenar el arreglo ni con tabla de hash**, sino puramente por división y conquista.

10. (★★★★) Resolver el ejercicio anterior, por división y conquista, en complejidad $\mathcal{O}(n)$, dada la misma aclaración. Justificar la complejidad de la solución.

11. (★★★★) Implementar una función, que utilice división y conquista, de complejidad $\mathcal{O}(n)$ que dado un arreglo de $n$ números enteros devuelva `true` o `false` según si existe algún elemento que aparezca más de dos tercios de las veces. Justificar la complejidad de la solución.

12. (★★★) Tenemos un arreglo de tamaño `2n` de la forma `{C1, C2, C3, … Cn, D1, D2, D3, … Dn}`, tal que la cantidad total de elementos del arreglo es potencia de 2 (por ende, `n` también lo es). Implementar un algoritmo de División y Conquista que modifique el arreglo de tal forma que quede con la forma `{C1, D1, C2, D2, C3, D3, …, Cn, Dn}`, **sin utilizar espacio adicional** (obviando el utilizado por la recursividad). Indicar y justificar su complejidad temporal.

    *Pista*: Pensar primero cómo habría que hacer si el arreglo tuviera 4 elementos (`{C1, C2, D1, D2}`). Luego, pensar a partir de allí el caso de 8 elementos, etc… para encontrar el patrón.

13. (★★) Debido a la trágica situación actual, es necesario realizar tests para detectar si alguna persona está contagiada de COVID-19. El problema es que los insumos tienden a ser bastante caros, y no vivimos en un país al que los recursos le sobren.

Supongamos que por persona se toma más de una muestra (lo cual es cierto, pero a fines del ejercicio supongamos que son muchas muestras), y que podemos realizar un testeo a más de una persona al mismo tiempo mezclando las muestras (lo cual también es cierto): determinamos un conjunto de personas a testear, obtenemos una muestra de cada una de ellas, las "juntamos", y al conjunto le realizamos el test. Si el test resulta negativo, implica que todas las personas testeadas en conjunto resultaron negativas. Si resulta positivo, implica que al menos una de las personas testedas resulta positiva.

Suponer que existe una función `pcr(grupo)`, que devuelve `true` si al menos una persona del `grupo` es COVID-positivo, y `false` en caso contrario (los `grupos` pueden estar formados por 1 o más personas). Suponer que la positividad es extremadamente baja, e inclusive pueden suponer que va a haber una única persona contagiada (por simplicidad).

Implementar un algoritmo que dado un conjunto de `n` personas, devuelva la o las personas contagiadas, utilizando la menor cantidad de tests posibles (considerando la notación Big Oh). En dicha notación, ¿cuántos tests se estarán utilizando?

Pueden considerar que habrá una única persona contagiada, pero esto no cambiará el análisis a realizar.

14. (★★) Se sabe, por el teorema de Bolzano, que si una función es continua en un intervalo `[a, b]`, y que en el punto `a` es positiva y en el punto `b` es negativa (o viceversa), necesariamente debe haber (al menos) una raíz en dicho intervalo. Implementar una función raiz que reciba una función (univariable) y los extremos mencionados a y b, y devuelva una raíz dentro de dicho intervalo (si hay más de una, simplemente quedarse con una). La complejidad de dicha función debe ser logarítmica del largo del intervalo `[a, b]`. Asumir que por más que se esté trabajando con números enteros, hay raíz en dichos valores: Se puede trabajar con `floats`, y el algoritmo será equivalente, simplemente se plantea con `ints` para no generar confusiones con la complejidad. Justificar la complejidad de la función implementada.

15. (★) Es el año 1700, y la pirata Barba-ra Verde atacó un barco de la *Royal British Shipping & Something*, que transportaba una importante piedra preciosa de la corona británica. Al parecer, la escondieron en un cofre con muchas piedras preciosas falsas, en caso de un ataque. Barba-ra Verde sabe que los refuerzos británicos no tardarán en llegar, y deben huir lo más rápido posible. El problema es que no pueden llevarse el cofre completo por pesar demasiado. Necesita encontrar rápidamente la joya verdadera. La única forma de descubrir la joya verdadera es pesando. Se sabe que la joya verdadera va a pesar más que las imitaciones, y que las imitaciones pesan todas lo mismo. Cuenta con una balanza de platillos para poder pesarlas (es el 1700, no esperen una balanza digital).

    a. Escribir un algoritmo de división y conquista, para determinar cuál es la verdadera joya de la corona. Suponer que hay una función `balanza(grupo_de_joyas1, grupo_de_joyas2)` que devuelve 0 si ambos grupos pesan lo mismo, mayor a 0 si el `grupo1` pesa más que el `grupo2`, y menor que 0 si pasa lo contrario, y realiza esto en tiempo constante. b. Indicar y justificar (adecuadamente) la complejidad de la función implementada.

16. (★★★★) Sea una matriz $A$ de tamaño $n \times n$, con todos valores distintos. Un índice $(i, j)$ es un máximo local si $A\,[i, j]$ es estrictamente mayor que todos su vecinos que existan (arriba, abajo, izquierda, derecha). Implementar un algoritmo de División y Conquista que permita encontrar algún máximo local en tiempo $\mathcal{O}(n)$. Justificar adecuadamente la complejidad del algoritmo. Prestar mucha atención a la ecuación de recurrencia escrita, ya que esto puede develar un error en el algoritmo planteado.

17. (★★★) Una función lineal se define como $f(x) = ax + b$. Si tenemos dos funciones lineales $f_1(x) = a_1x + b_1$ y $f_2(x) = a_2x + b_2$, entonces la composición puede simplificarse a: $f_3(x) = f_1(f_2(x)) = (a_1a_2)\,x + (a_1b_2 + b_1)$. Es decir, tenemos una nueva función lineal, cuyos $a$ y $b$ son los resultates marcados.

    Utilizando **división y conquista**, implementar una función `composición_n(a, b, c, n)` que reciba los valores de $a$ y $b$ de una función lineal, `c` y `n` y determine el valor de $f^n(c) = f(f(f(...f(c))$ ($n$ composiciones de la función $f$ consigo misma) en tiempo $\mathcal{O}(\log n)$. Justificar adecuadamente la complejidad del algoritmo implementado.

    Ayudas:

    - Recomendamos primero obtener los valores de $a$ y $b$ que corresponden a $f^n(x)$. La cuenta final es trivial.

▶ Quiero unas ayudas más